

Tutorial #6 — Python Fundamentals for Data Science

Exercise Solutions

Topics: Variables, Data Types, Strings, Lists, Tuples, Dictionaries, Sets, Indexing & Slicing, List Comprehension, Control Flow, Functions, Lambda Functions, `map()`, `filter()`, `sorted()`, and Basic File Handling.

Note: These solutions are based on the exercises provided in Tutorial #6. The notebook is organized with clear section headings and executable Python cells.

1. Variables

Exercise 1: Student Information

Create variables to store Student name, Roll number, Age, Department, Semester, and CGPA. Display all information in a meaningful format.

```
# Student Information
student_name = "Rahul Sharma"
roll_number = 101
age = 21
department = "CSE"
semester = 5
cgpa = 8.5

# Display all information in a meaningful format
print(f"Student Name: {student_name}")
print(f"Roll Number: {roll_number}")
print(f"Age: {age}")
print(f"Department: {department}")
print(f"Semester: {semester}")
print(f"CGPA: {cgpa}")
```

```
Student Name: Rahul Sharma
Roll Number: 101
Age: 21
Department: CSE
Semester: 5
CGPA: 8.5
```

Exercise 2: Employee Salary

Create variables for Basic Salary = ₹35,000, HRA = ₹8,000, and DA = ₹5,000. Calculate and display the gross salary.

```
# Employee Salary
basic_salary = 35000
hra = 8000
da = 5000

# Calculate gross salary
gross_salary = basic_salary + hra + da

# Display the result
print(f"Gross Salary: ₹ {gross_salary}")

Gross Salary: ₹ 48000
```

Exercise 3: Temperature Conversion

Store temperature in Celsius in a variable and convert it to Fahrenheit.

Fahrenheit = (Celsius × 9/5) + 32

```
# Temperature Conversion
celsius = 25

# Convert to Fahrenheit
fahrenheit = (celsius * 9/5) + 32

# Display the results
print(f"Temperature in Celsius: {celsius}")
print(f"Temperature in Fahrenheit: {fahrenheit}")

Temperature in Celsius: 25
Temperature in Fahrenheit: 77.0
```

2. Data Types

Exercise 1: Identify Data Types

age = 21 percentage = 87.5 name = "Rahul" is_passed = True

Use type() to identify the data type of each variable.

```
age = 21
percentage = 87.5
```

```
name = "Rahul"
is_passed = True

# Use type() to identify the data type of each variable
print(f"age: {type(age)}")
print(f"percentage: {type(percentage)}")
print(f"name: {type(name)}")
print(f"is_passed: {type(is_passed)}")

age: <class 'int'>
percentage: <class 'float'>
name: <class 'str'>
is_passed: <class 'bool'>
```

Exercise 2: Type Conversion

Given marks = "85", convert it into integer, float, and string. Display the value and data type after each conversion.

```
marks = "85"

# Convert to integer
marks_int = int(marks)
print(marks_int, type(marks_int))

# Convert to float
marks_float = float(marks)
print(marks_float, type(marks_float))

# Convert to string (it's already a string, but converting explicitly)
marks_str = str(marks)
print(marks_str, type(marks_str))

85 <class 'int'>
85.0 <class 'float'>
85 <class 'str'>
```

Exercise 3: Mixed Data

Create variables representing number of students, average marks, college name, and whether the class is active. Display each variable with its data type.

```
# Mixed Data
num_students = 60
average_marks = 78.5
college_name = "ABC College"
class_active = True

# Display each variable with its data type
```

```

print(f"Number of Students : {num_students} | Type:
{type(num_students).__name__}")
print(f"Average Marks : {average_marks} | Type:
{type(average_marks).__name__}")
print(f"College Name : {college_name} | Type:
{type(college_name).__name__}")
print(f"Class Active : {class_active} | Type:
{type(class_active).__name__}")

```

```

Number of Students : 60 | Type: int
Average Marks : 78.5 | Type: float
College Name : ABC College | Type: str
Class Active : True | Type: bool

```

3. Strings

Exercise 1: Student Name

```
name = "Rahul Sharma"
```

Display the complete name, first character, last character, length, uppercase version, and lowercase version.

```

name = "Rahul Sharma"

# Display the complete name, first character, last character, length,
uppercase, lowercase
print(f"Complete name: {name}")
print(f"First character: {name[0]}")
print(f>Last character: {name[-1]}")
print(f"Length: {len(name)}")
print(f"Uppercase: {name.upper()}")
print(f"Lowercase: {name.lower()}")

```

```

Complete name: Rahul Sharma
First character: R
Last character: a
Length: 12
Uppercase: RAHUL SHARMA
Lowercase: rahul sharma

```

Exercise 2: Data Cleaning

```
email = "    STUDENT@GMAIL.COM    "
```

Remove leading/trailing spaces and convert the email to lowercase. Expected result:
student@gmail.com

```
email = "    STUDENT@GMAIL.COM    "
clean_email = email.strip().lower()
print("Cleaned email:", clean_email)
Cleaned email: student@gmail.com
```

Exercise 3: String Analysis

```
text = "    Python is useful for Data Science    "
```

Find the number of characters, number of words, occurrences of 'Data', and whether 'Python' exists in the string.

```
text = "Python is useful for Data Science"

num_characters = len(text)
num_words = len(text.split())
occurrences_data = text.count('Data')
is_python_present = 'Python' in text

print(f"Number of characters: {num_characters}")
print(f"Number of words: {num_words}")
print(f"Occurrences of 'Data': {occurrences_data}")
print(f"Is 'Python' present?: {is_python_present}")

Number of characters: 33
Number of words: 6
Occurrences of 'Data': 1
Is 'Python' present?: True
```

4. Lists

Exercise 1: Student Marks

```
marks = [78, 85, 62, 90, 55, 71]
```

Display the list, number of students, maximum marks, minimum marks, total marks, and average marks.

```
marks = [78, 85, 62, 90, 55, 71]

num_students = len(marks)
max_marks = max(marks)
min_marks = min(marks)
total_marks = sum(marks)
```

```

average_marks = total_marks / num_students

print(f"Marks: {marks}")
print(f"Number of students: {num_students}")
print(f"Maximum: {max_marks}")
print(f"Minimum: {min_marks}")
print(f"Total: {total_marks}")
print(f"Average: {average_marks}")

```

```

Marks: [78, 85, 62, 90, 55, 71]
Number of students: 6
Maximum: 90
Minimum: 55
Total: 441
Average: 73.5

```

Exercise 2: Modify a List

```
subjects = ["Python", "DBMS", "OS", "Computer Networks"]
```

Add Machine Learning; insert Statistics at position 2; remove OS; change DBMS to Database Management; display the final list.

```

subjects = ["Python", "DBMS", "OS", "Computer Networks"]

# Add Machine Learning
subjects.append("Machine Learning")

# Insert Statistics at position 2
subjects.insert(2, "Statistics")

# Remove OS
subjects.remove("OS")

# Change DBMS to Database Management
subjects[subjects.index("DBMS")] = "Database Management"

print(subjects)

['Python', 'Database Management', 'Statistics', 'Computer Networks',
'Machine Learning']

```

Exercise 3: Data Collection

Create a list containing 10 student marks. Perform append(), insert(), remove(), pop(), sort(), and reverse(). Observe the changes.

```
marks = [78, 65, 90, 55, 82, 71, 68, 95, 73, 88]
```

```

??
print("After append:", marks)

??
print("After insert:", marks)

??
print("After remove:", marks)

??
print("Popped value:", removed)
print("After pop:", marks)

??
print("After sort:", marks)

??
print("After reverse:", marks)

After append: [78, 65, 90, 55, 82, 71, 68, 95, 73, 88, 91]
After insert: [78, 65, 80, 90, 55, 82, 71, 68, 95, 73, 88, 91]
After remove: [78, 65, 80, 90, 82, 71, 68, 95, 73, 88, 91]
Popped value: 90
After pop: [78, 65, 80, 82, 71, 68, 95, 73, 88, 91]
After sort: [65, 68, 71, 73, 78, 80, 82, 88, 91, 95]
After reverse: [95, 91, 88, 82, 80, 78, 73, 71, 68, 65]

```

5. Tuples

Exercise 1: Student Record

```
student = ("Rahul", 101, "CSE", 8.5)
```

Display student name, roll number, department, and CGPA.

```

student = ("Rahul", 101, "CSE", 8.5)

print(f"Student Name: {student[0]}")
print(f"Roll Number: {student[1]}")
print(f"Department: {student[2]}")
print(f"CGPA: {student[3]}")

Student Name: Rahul
Roll Number: 101
Department: CSE
CGPA: 8.5

```

Exercise 2: Coordinates

```
point = (10, 20, 30)
```

Display each coordinate, find tuple length, maximum value, and minimum value.

```
marks = [35, 67, 89, 42, 90, 28, 76, 55]

passed = [m for m in marks if m >= 40]
failed = [m for m in marks if m < 40]
above_75 = [m for m in marks if m > 75]
increased_marks = [m + 5 for m in marks]

print("Passed:", passed)
print("Failed:", failed)
print("Above 75:", above_75)
print("Marks increased by 5:", increased_marks)

Passed: [67, 89, 42, 90, 76, 55]
Failed: [35, 28]
Above 75: [89, 90, 76]
Marks increased by 5: [40, 72, 94, 47, 95, 33, 81, 60]
```

Exercise 3: Tuple Immutability

```
subjects = ("Python", "Statistics", "Machine Learning")
```

Try to change Statistics to Data Science. Observe the error and explain why tuples cannot be modified.

```
subjects = ("Python", "Statistics", "Machine Learning")

print("Original tuple:", subjects)

# Try to change 'Statistics' to 'Data Science'
try:
    subjects[1] = "Data Science"
except TypeError:
    print("Tuples cannot be modified after creation.")

Original tuple: ('Python', 'Statistics', 'Machine Learning')
Tuples cannot be modified after creation.
```

6. Dictionaries

Exercise 1: Student Dictionary

```
student = {  
    "name": "Rahul",  
    "roll_no": 101,  
    "marks": 85,  
    "department": "CSE"  
}
```

Display name and marks; add city = Pune; change marks to 92; remove roll_no; display all keys and values.

```
student = {  
    "name": "Rahul",  
    "roll_no": 101,  
    "marks": 85,  
    "department": "CSE"  
}  
  
# Display name and marks  
print("Name:", student["name"])  
print("Marks:", student["marks"])  
  
# Add city = Pune  
student["city"] = "Pune"  
  
# Change marks to 92  
student["marks"] = 92  
  
# Remove roll_no  
del student["roll_no"]  
  
# Display all keys and values  
print("Final dictionary:", student)  
print("Keys:", student.keys())  
print("Values:", student.values())  
  
Name: Rahul  
Marks: 85  
Final dictionary: {'name': 'Rahul', 'marks': 92, 'department': 'CSE',  
                  'city': 'Pune'}  
Keys: dict_keys(['name', 'marks', 'department', 'city'])  
Values: dict_values(['Rahul', 92, 'CSE', 'Pune'])
```

Exercise 2: Employee Data

Create a dictionary containing Name, Employee ID, Department, Salary, and Experience. Display the employee information.

```
employee = {
    "Name": "Amit Sharma",
    "Employee ID": "EMP101",
    "Department": "Data Science",
    "Salary": 65000,
    "Experience": 4
}

for key, value in employee.items():
    print(f"{key} : {value}")

Name : Amit Sharma
Employee ID : EMP101
Department : Data Science
Salary : 65000
Experience : 4
```

Exercise 3: Subject Marks

```
marks = {
    "Python": 85,
    "Statistics": 78,
    "Machine Learning": 91,
    "DBMS": 74
}
```

Find total marks, average marks, highest marks, and the subject with the highest marks.

```
marks = {
    "Python": 85,
    "Statistics": 78,
    "Machine Learning": 91,
    "DBMS": 74
}

total = sum(marks.values())
average = total / len(marks)
highest_mark = max(marks.values())
highest_subject = max(marks, key=marks.get)

print("Total:", total)
print("Average:", average)
print("Highest Mark:", highest_mark)
print("Highest-scoring Subject:", highest_subject)
```

```
Total: 328
Average: 82.0
Highest Mark: 91
Highest-scoring Subject: Machine Learning
```

7. Sets

Exercise 1: Remove Duplicates

```
numbers = [10, 20, 10, 30, 20, 40, 30, 50]
```

Convert the list into a set to obtain only unique values.

```
numbers = [10, 20, 10, 30, 20, 40, 30, 50]
unique_values = set(numbers)

print("Original list:", numbers)
print("Unique values:", unique_values)

Original list: [10, 20, 10, 30, 20, 40, 30, 50]
Unique values: {40, 10, 50, 20, 30}
```

Exercise 2: Common Subjects

```
student1 = {"Python", "DBMS", "OS", "Statistics"}
student2 = {"Python", "Machine Learning", "Statistics", "AI"}
```

Find subjects common to both students, all subjects selected by either student, subjects selected only by Student 1, and subjects selected only by Student 2.

```
student1 = {"Python", "DBMS", "OS", "Statistics"}
student2 = {"Python", "Machine Learning", "Statistics", "AI"}

common_subjects = student1 & student2
all_subjects = student1 | student2
only_student1 = student1 - student2
only_student2 = student2 - student1

print("Common subjects:", common_subjects)
print("All subjects:", all_subjects)
print("Only Student 1:", only_student1)
print("Only Student 2:", only_student2)

Common subjects: {'Python', 'Statistics'}
All subjects: {'Python', 'OS', 'DBMS', 'AI', 'Machine Learning', 'Statistics'}
```

```
Only Student 1: {'OS', 'DBMS'}  
Only Student 2: {'Machine Learning', 'AI'}
```

Exercise 3: Unique Categories

```
categories = ["AI", "ML", "AI", "Data Science", "ML", "Python", "AI"]
```

Find the unique categories.

```
categories = ["AI", "ML", "AI", "Data Science", "ML", "Python", "AI"]  
unique_categories = set(categories)  
print("Unique categories:", unique_categories)  
Unique categories: {'ML', 'AI', 'Python', 'Data Science'}
```

8. Indexing and Slicing

Exercise 1: List Indexing

```
marks = [78, 85, 62, 90, 55, 71, 88]
```

Display the first element, last element, second element, second-last element, and third element.

```
marks = [78, 85, 62, 90, 55, 71, 88]  
print("First element:", marks[0])  
print("Last element:", marks[-1])  
print("Second element:", marks[1])  
print("Second-last element:", marks[-2])  
print("Third element:", marks[2])
```

```
First element: 78  
Last element: 88  
Second element: 85  
Second-last element: 71  
Third element: 62
```

Exercise 2: List Slicing

Using the same list, extract the first three marks, last three marks, elements from index 2 to 5, every second element, and the list in reverse order.

```
marks = [78, 85, 62, 90, 55, 71, 88]

first_three = marks[:3]
last_three = marks[-3:]
index_2_to_5 = marks[2:5]
every_second = marks[::2]
reverse = marks[::-1]

print("First three:", first_three)
print("Last three:", last_three)
print("Index 2 to 5:", index_2_to_5)
print("Every second element:", every_second)
print("Reverse:", reverse)
```

First three: [78, 85, 62]
 Last three: [55, 71, 88]
 Index 2 to 5: [62, 90, 55]
 Every second element: [78, 62, 55, 88]
 Reverse: [88, 71, 55, 90, 62, 85, 78]

Exercise 3: String Slicing

```
course = "DataScience"
```

Extract Data and Science. Also display the string in reverse order.

```
course = "DataScience"

data_part = course[0:4]
science_part = course[4:11]
reverse_course = course[::-1]

print("Data:", data_part)
print("Science:", science_part)
print("Reverse:", reverse_course)
```

Data: Data
 Science: Science
 Reverse: ecneicSataD

9. List Comprehension

Exercise 1: Squares

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Create a list containing the squares using list comprehension.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = [n**2 for n in numbers]

print("Numbers:", numbers)
print("Squares:", squares)

Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Exercise 2: Even Numbers

```
numbers = [10, 15, 20, 25, 30, 35, 40]
```

Create a list containing only even numbers.

```
numbers = [10, 15, 20, 25, 30, 35, 40]
even_numbers = [n for n in numbers if n % 2 == 0]
print(even_numbers)

[10, 20, 30, 40]
```

Exercise 3: Passing Students

```
marks = [35, 67, 89, 42, 90, 28, 76, 55]
```

Create lists of passing marks, failing marks, marks greater than 75, and marks increased by 5.

```
marks = [35, 67, 89, 42, 90, 28, 76, 55]

passed = [m for m in marks if m >= 40]
failed = [m for m in marks if m < 40]
above_75 = [m for m in marks if m > 75]
increased_marks = [m + 5 for m in marks]

print("Passed:", passed)
print("Failed:", failed)
print("Above 75:", above_75)
print("Marks increased by 5:", increased_marks)

Passed: [67, 89, 42, 90, 76, 55]
Failed: [35, 28]
Above 75: [89, 90, 76]
Marks increased by 5: [40, 72, 94, 47, 95, 33, 81, 60]
```

Exercise 4: Data Transformation

```
temperatures = [20, 25, 30, 35, 40]
```

Convert each temperature from Celsius to Fahrenheit using list comprehension.

```
temperatures = [20, 25, 30, 35, 40]
fahrenheit = [(c * 9/5) + 32 for c in temperatures]

print("Celsius:", temperatures)
print("Fahrenheit:", fahrenheit)

Celsius: [20, 25, 30, 35, 40]
Fahrenheit: [68.0, 77.0, 86.0, 95.0, 104.0]
```

10. Control Flow

Exercise 1: Pass or Fail

Given a student's marks, display Pass if marks are ≥ 40 , otherwise display Fail.

```
marks = 45

if marks >= 40:
    print("Pass")
else:
    print("Fail")

Pass
```

Exercise 2: Grade Calculator

Assign grades: 90–100 A+, 80–89 A, 70–79 B, 60–69 C, 40–59 D, below 40 F.

```
marks = 85

if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
    grade = "D"
```

```
else:  
    grade = "F"  
  
print("Grade:", grade)
```

Grade: A

Exercise 3: Even and Odd

```
numbers = [10, 15, 22, 31, 40, 51, 60]
```

Use a loop and if-else to display whether each number is even or odd.

```
numbers = [10, 15, 22, 31, 40, 51, 60]  
  
for num in numbers:  
    if num % 2 == 0:  
        print(num, "is Even")  
    else:  
        print(num, "is Odd")
```

```
10 is Even  
15 is Odd  
22 is Even  
31 is Odd  
40 is Even  
51 is Odd  
60 is Even
```

Exercise 4: Maximum Without max()

```
numbers = [45, 78, 23, 91, 56]
```

Use a for loop and conditional statements to find the largest number without using max().

```
numbers = [45, 78, 23, 91, 56]  
  
largest = numbers[0]  
for num in numbers:  
    if num > largest:  
        largest = num  
  
print("Largest number:", largest)
```

Largest number: 91

Exercise 5: break and continue

```
numbers = [10, 20, 30, 40, 50, 60]
```

Stop processing when the value becomes 40 using break. Then write another program that skips 40 using continue.

```
numbers = [10, 20, 30, 40, 50, 60]
```

```
print("Using break:")
for num in numbers:
    if num == 40:
        break
    print(num)
```

```
print("\nUsing continue:")
for num in numbers:
    if num == 40:
        continue
    print(num)
```

Using break:

```
10
20
30
```

Using continue:

```
10
20
30
50
60
```

11. Functions

Exercise 1: Calculate Average

Create `calculate_average(marks)` that accepts a list of marks and returns the average.

```
def calculate_average(marks):
    average = sum(marks) / len(marks)
    return average

marks = [78, 85, 62, 90, 55, 71, 88]
print("Average:", calculate_average(marks))

Average: 75.57142857142857
```

Exercise 2: Statistics Function

Create `calculate_statistics(numbers)` that returns minimum, maximum, sum, and average.

```
def calculate_statistics(numbers):
    return {
        "Minimum": min(numbers),
        "Maximum": max(numbers),
        "Sum": sum(numbers),
        "Average": sum(numbers) / len(numbers)
    }

numbers = [10, 20, 30, 40, 50]
statistics = calculate_statistics(numbers)
for key, value in statistics.items():
    print(key, ":", value)

Minimum : 10
Maximum : 50
Sum : 150
Average : 30.0
```

Exercise 3: Grade Function

Create `calculate_grade(marks)` that returns the appropriate grade.

```
def calculate_grade(marks):
    if marks >= 90:
        return "A+"
    elif marks >= 80:
        return "A"
    elif marks >= 70:
        return "B"
    elif marks >= 60:
        return "C"
    elif marks >= 40:
        return "D"
    else:
        return "F"

marks = 85
print("Grade:", calculate_grade(marks))

Grade: A
```

Exercise 4: Temperature Function

Create `celsius_to_fahrenheit(celsius)` and use it to convert 0°C, 25°C, 37°C, and 100°C.

```
def celsius_to_fahrenheit(celsius):
    fahrenheit = (celsius * 9/5) + 32
    return fahrenheit
```

```
temperatures = [0, 25, 37, 100]
for celsius in temperatures:
    print(celsius, "°C =", celsius_to_fahrenheit(celsius), "°F")

0 °C = 32.0 °F
25 °C = 77.0 °F
37 °C = 98.6 °F
100 °C = 212.0 °F
```

Exercise 5: Count Passing Students

Create `count_passing_students(marks)` that returns the number of students who passed.

```
def count_passing_students(marks):
    count = 0
    for mark in marks:
        if mark >= 40:
            count += 1
    return count

marks = [35, 67, 89, 42, 90, 28, 76, 55]
print("Number of passing students:", count_passing_students(marks))

Number of passing students: 6
```

12. Lambda Functions

- Create a lambda function to calculate the square of a number.
- Create a lambda function to calculate the cube of a number.
- Create a lambda function that returns the larger of two numbers.
- Create a lambda function that returns "Pass" if marks ≥ 40 , otherwise "Fail".
- Create a lambda function that converts marks out of 500 into percentage.

```
# 1. Square
square = lambda x: x ** 2
print("Square:", square(5))

# 2. Cube
cube = lambda x: x ** 3
print("Cube:", cube(5))

# 3. Larger of two numbers
larger = lambda a, b: a if a > b else b
print("Larger:", larger(10, 25))

# 4. Pass / Fail
```

```
pass_fail = lambda marks: "Pass" if marks >= 40 else "Fail"
print("Result:", pass_fail(65))

# 5. Percentage out of 500
percentage = lambda marks: (marks / 500) * 100
print("Percentage:", percentage(425))

Square: 25
Cube: 125
Larger: 25
Result: Pass
Percentage: 85.0
```

13. `map()`, `filter()`, `sorted()`

A. `map()` —

Exercise 1: Squares

```
numbers = [1, 2, 3, 4, 5]
```

Use `map()` and `lambda` to generate `[1, 4, 9, 16, 25]`.

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
print(squares)

[1, 4, 9, 16, 25]
```

Exercise 2: Celsius to Fahrenheit

```
temperatures = [0, 10, 20, 30, 40]
```

Use `map()` to convert all temperatures to Fahrenheit.

```
temperatures = [0, 10, 20, 30, 40]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, temperatures))
print(fahrenheit)

[32.0, 50.0, 68.0, 86.0, 104.0]
```

B. filter() — Exercise 3: Even Numbers

```
numbers = [10, 15, 20, 25, 30, 35, 40]
```

Use filter() to obtain only even numbers.

```
numbers = [10, 15, 20, 25, 30, 35, 40]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)
[10, 20, 30, 40]
```

Exercise 4: Passing Marks

```
marks = [35, 67, 89, 42, 90, 28, 76, 55]
```

Use filter() to obtain marks ≥ 40 .

```
marks = [35, 67, 89, 42, 90, 28, 76, 55]
passed = list(filter(lambda m: m >= 40, marks))
print(passed)
[67, 89, 42, 90, 76, 55]
```

C. sorted() — Exercise 5: Sort Marks

```
marks = [78, 45, 90, 62, 88, 55]
```

Sort the marks in ascending and descending order.

```
marks = [78, 45, 90, 62, 88, 55]
ascending = sorted(marks)
descending = sorted(marks, reverse=True)
print("Ascending:", ascending)
print("Descending:", descending)
Ascending: [45, 55, 62, 78, 88, 90]
Descending: [90, 88, 78, 62, 55, 45]
```

Exercise 6: Sort Students by Marks

```
students = [
    ("Rahul", 85),
```

```
("Amit", 72),  
("Priya", 92),  
("Sneha", 78)  
]
```

Sort students according to their marks.

```
students = [  
    ("Rahul", 85),  
    ("Amit", 72),  
    ("Priya", 92),  
    ("Sneha", 78)  
]  
  
students_by_marks = sorted(students, key=lambda student: student[1])  
print(students_by_marks)  
[('Amit', 72), ('Sneha', 78), ('Rahul', 85), ('Priya', 92)]
```

Exercise 7: Sort by Name

Using the same data, sort students alphabetically according to their names.

```
students = [  
    ("Rahul", 85),  
    ("Amit", 72),  
    ("Priya", 92),  
    ("Sneha", 78)  
]  
  
students_by_name = sorted(students, key=lambda student: student[0])  
print(students_by_name)  
[('Amit', 72), ('Priya', 92), ('Rahul', 85), ('Sneha', 78)]
```

14. Basic File Handling

Exercise 1: Opening a File

Create students.txt. Open it using open() and display whether the file was opened successfully.

```
file = open("students.txt", "w")  
file.close()
```

```
file = open("students.txt", "r")
print("File opened successfully:", not file.closed)
file.close()
```

File opened successfully: True

Exercise 2: Read Entire File

Create a text file containing Rahul, Amit, Priya, and Sneha on separate lines. Write a Python program to read and display the complete file.

```
# Create the file with student names on separate lines
with open("students.txt", "w") as file:
    file.write("Rahul\nAmit\nPriya\nSneha\n")

# Read and display the complete file
with open("students.txt", "r") as file:
    content = file.read()

print(content)
```

Rahul
Amit
Priya
Sneha

Exercise 3: Read Line by Line

Read the same file line by line using readline() or a for loop. Display each student name separately.

```
# Read line by line using a for loop
with open("students.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Rahul
Amit
Priya
Sneha

Exercise 4: Count Lines

Read a text file containing student names and count the total number of students.

```
with open("students.txt", "r") as file:
    student_count = sum(1 for line in file)
```

```
print("Number of students:", student_count)
```

Number of students: 4

Exercise 5: Create a Student File

Create students.txt and write the following data into it:

```
Rahul,85  
Amit,72  
Priya,91  
Sneha,78
```

```
with open("students.txt", "w") as file:  
    file.write("Rahul,85\n")  
    file.write("Amit,72\n")  
    file.write("Priya,91\n")  
    file.write("Sneha,78\n")
```

```
# Optional: verify by reading it back  
with open("students.txt", "r") as file:  
    print(file.read())
```

```
Rahul,85  
Amit,72  
Priya,91  
Sneha,78
```

Exercise 6: Read Using with open()

```
with open("students.txt", "r") as file:  
    # read and display the contents
```

Read the file using with open(). Explain why this approach is preferred over manually calling close().

```
with open("students.txt", "r") as file:  
    content = file.read()
```

```
print(content)
```

```
Rahul,85  
Amit,72  
Priya,91  
Sneha,78
```

Why is with open() preferred? It automatically closes the file when the block finishes, even if an exception occurs. This makes file handling safer and cleaner.

Exercise 7: Write Mode (w)

Open a file using w mode and write Python, NumPy, Pandas, and Machine Learning. Observe what happens if the file already contains data.

```
with open("subjects.txt", "w") as file:
    file.write("Python\n")
    file.write("NumPy\n")
    file.write("Pandas\n")
    file.write("Machine Learning\n")

with open("subjects.txt", "r") as file:
    print(file.read())
```

```
Python
NumPy
Pandas
Machine Learning
```

Observation: w mode creates a file if it does not exist and overwrites existing content.

Exercise 8: Append Mode (a)

Open the same file using a mode and add Data Visualization and Statistics. Check the file contents.

```
with open("subjects.txt", "a") as file:
    file.write("Data Visualization\n")
    file.write("Statistics\n")

with open("subjects.txt", "r") as file:
    print(file.read())
```

```
Python
NumPy
Pandas
Machine Learning
Data Visualization
Statistics
```

Observation: a mode adds new content to the end of the existing file without overwriting it.

Exercise 9: Read and Process Data

```
marks.txt:
85
72
91
```

```
67
78
90
```

Read all marks, convert them from strings to integers, calculate total and average, and find the highest and lowest mark.

```
# Create the marks.txt file
with open("marks.txt", "w") as file:
    file.write("85\n72\n91\n67\n78\n90\n")

# Read all marks, convert to integers, and process
with open("marks.txt", "r") as file:
    marks = [int(line.strip()) for line in file]

total = sum(marks)
average = total / len(marks)
highest = max(marks)
lowest = min(marks)

print("Marks:", marks)
print("Total:", total)
print("Average:", average)
print("Highest:", highest)
print("Lowest:", lowest)

Marks: [85, 72, 91, 67, 78, 90]
Total: 483
Average: 80.5
Highest: 91
Lowest: 67
```

15. Integrated Exercise: Student Marks Data Processing (Optional)

Create student_marks.txt containing:

```
Rahul,85
Amit,72
Priya,91
Sneha,38
Karan,67
Neha,95
```

Write a Python program that:

- Opens the file using with open().
- Reads the file.
- Separates student names and marks.
- Stores the information in a dictionary.

- Finds the average marks.
- Finds the highest-scoring student.
- Finds the lowest-scoring student.
- Identifies students who passed.
- Identifies students who failed.
- Sorts students according to their marks.
- Writes the final result to result.txt.

Expected result should include:

Average Marks: 74.67

Highest Scorer: Neha - 95

Lowest Scorer: Sneha - 38

Passed Students:

Rahul
Amit
Priya
Karan
Neha

Failed Students:

Sneha

Step 1: Create student_marks.txt

```
with open("student_marks.txt", "w") as file:
    file.write("Rahul,85\n")
    file.write("Amit,72\n")
    file.write("Priya,91\n")
    file.write("Sneha,38\n")
    file.write("Karan,67\n")
    file.write("Neha,95\n")
```

Step 2: Open and read the file, separating names and marks into a dictionary

```
students = {}
with open("student_marks.txt", "r") as file:
    for line in file:
        name, marks = line.strip().split(",")
        students[name] = int(marks)
```

Step 3: Find average marks

```
average_marks = sum(students.values()) / len(students)
```

Step 4: Find highest-scoring student

```
highest_scorer = max(students, key=students.get)
highest_marks = students[highest_scorer]
```

Step 5: Find lowest-scoring student

```
lowest_scorer = min(students, key=students.get)
lowest_marks = students[lowest_scorer]
```

```

# Step 6: Identify passed and failed students (passing mark >= 40)
passed_students = [name for name, marks in students.items() if marks
>= 40]
failed_students = [name for name, marks in students.items() if marks <
40]

# Step 7: Sort students according to their marks (ascending)
sorted_students = sorted(students.items(), key=lambda item: item[1])

# Step 8: Write the final result to result.txt
with open("result.txt", "w") as file:
    file.write(f"Average Marks: {average_marks:.2f}\n")
    file.write(f"Highest Scorer: {highest_scorer} - {highest_marks}\n")
    file.write(f"Lowest Scorer: {lowest_scorer} - {lowest_marks}\n\n")

    file.write("Passed Students:\n")
    for name in passed_students:
        file.write(f"{name}\n")

    file.write("\nFailed Students:\n")
    for name in failed_students:
        file.write(f"{name}\n")

    file.write("\nStudents Sorted by Marks:\n")
    for name, marks in sorted_students:
        file.write(f"{name}: {marks}\n")

# Step 9: Display the result by reading result.txt back
with open("result.txt", "r") as file:
    print(file.read())

```

16. Summary

This notebook provides exercises in Tutorial #6, covering:

- Variables and data types
- Strings
- Lists and tuples
- Dictionaries and sets
- Indexing and slicing
- List comprehension
- Control flow
- Functions
- Lambda functions
- `map()`, `filter()`, and `sorted()`

- Basic file handling with `r`, `w`, `a`, and `with open()`
- Integrated student marks data processing

The final integrated exercise connects Python fundamentals with simple **data manipulation and preprocessing**, providing a foundation for NumPy, Pandas, and later Data Science topics.

